

Eijen: Revised Padding and Instance-Specific Finalization

Eijen Algorithm Group

September 21, 2026

Abstract

This note specifies the revised pad_{10*1} rule and the instance-specific final capacity constant for the Eijen hash-function family. These changes are intended to prevent the identified non-random relation between Eijen instances with different digest lengths. We also correct the padding implementation for bit-aligned messages and regenerate the affected materials. Finally, we describe the current implementation and report performance measurements for the submitted version.

1 Revised Padding and Serialization

The original Eijen document specifies $M\|1\|0^k$ (pad_{10*}), whereas its appendix's **abc** example shows a terminal 1 in the final rate word. The implementation omits this terminal rate bit, creating an inconsistency between the specification, example, and code. The padding, byte-order description, test vectors, and implementations have therefore been revised.

The revised specification adopts little-endian serialization and complete rate-side pad_{10*1} padding, as used in SHA-3. It replaces the final capacity constant 1_c with the instance-specific D_h . The permutation, parameter sets, zero IV, and digest extraction positions remain unchanged. The new digests are incompatible with earlier versions, and the corresponding test vectors have been updated.

Let $b = 2048$, $c = h + 64$, and $r = b - c = 1984 - h$, with $h \in \{256, 384, 512, 768, 1024\}$. For an m -bit message, the revision uses the pad_{10*1} rule of FIPS 202, Algorithm 9:

$$j = (-m - 2) \bmod r, \quad 0 \leq j < r, \quad P_r(M) = M\|1\|0^j\|1, \quad \ell = \left\lceil \frac{m+2}{r} \right\rceil. \quad (1)$$

Message bits are consumed from low to high within each byte. If $q = m \bmod 8 \neq 0$, only the low q bits of the final input byte are valid; its high bits must be ignored. Each consecutive group of eight bytes forms a little-endian 64-bit word, and digest words are output in little-endian byte order. The first padding bit is at global bit position m and the second at $\ell r - 1$. When processing the last rate block, the capacity-side constant D_h is also injected into the final state word, with numeric value $2^{63} + h$. Capacity feed-forward uses the capacity input *after* this injection. No 64-bit or 128-bit message-length field is added.

1.1 Padding Examples

The following rate blocks are written in input-byte order; 00^k denotes k zero bytes. The bytes 01 and 80 mark the first and last padding bits. NULL means an empty message, not the four bytes of the string "NULL".

1. **Eijen-256, empty message.** Here $r = 1728$ bits (216 bytes), $m = 0$, and $j = 1726$. The single rate block is $01\|00^{214}\|80$. The padding bits are at global positions 0 and 1727.
2. **Eijen-256, abc.** The message bytes are 61 62 63; $m = 24$ and $j = 1702$. The single rate block is $61\ 62\ 63\ 01\|00^{211}\|80$. The padding bits are at positions 24 and 1727.
3. **Eijen-512, abc.** Here $r = 1472$ bits (184 bytes), $m = 24$, and $j = 1446$. The single rate block is $61\ 62\ 63\ 01\|00^{179}\|80$. The padding bits are at positions 24 and 1471.
4. **Eijen-512, two-block boundary.** Take an all-zero message of $m = r - 1 = 1471$ bits (183 zero bytes followed by a byte whose seven valid low bits are zero). Then $j = 1471$, and the two rate blocks are, in order, $00^{183}\|80$ and $00^{183}\|80$. The first padding bit is at the end of block 1 (global position 1471); the last is at the end of block 2 (position 2943). The capacity-side D_{512} is injected only with the final block.

1.2 Final Capacity Injection and Feed-Forward

The tag occupies absolute state word $A[31]$, which is in the capacity of all five instances. Its low 16 bits encode h as an unsigned integer; bits 16–62 are zero, and bit 63 retains the final-block marker. All other words of D_h are zero.

Instance	Final capacity word (numeric value)
Eijen-256	8000000000000100
Eijen-384	8000000000000180
Eijen-512	8000000000000200
Eijen-768	8000000000000300
Eijen-1024	8000000000000400

For the final padded rate block $M[\ell - 1]$, compute

$$\begin{aligned}
S &= X_{\ell-1} \oplus (M[\ell - 1] \parallel D_h), \\
C_{\text{saved}} &= \text{right}_c(S), \\
X_\ell &= \Pi(S) \oplus (0^r \parallel C_{\text{saved}}), \\
H_h(M) &= \text{right}_h(X_\ell).
\end{aligned}$$

Thus D_h affects both the permutation input and capacity feed-forward. It must not be applied only after the permutation. Non-final blocks retain their previous processing without D_h injection. The rate-side terminal 80 is unchanged. For Eijen-512, the final capacity word is encoded as the bytes 00 02 00 00 00 00 00 80.

2 Updated Materials

The revisions to little-endian serialization, padding, and finalization are reflected in the specification, the five reference implementations, the optimized implementations, and the four classes of official test vectors. The revised specification is `Eijen-pad10star1-domainsep.pdf`. The reference sources are in `Implementations/Reference_Implementation/`, and the AVX2 single-message sources are in `Implementations/Optimized_Implementation_AVX256_Single/`. The full official KATs are in `Test_Vectors/`; the 45 additional padding-boundary vectors are in `Test_Vectors/padding_note_vectors.json`. The single-message code uses the same AVX2 permutation path while omitting the four-message and AVX-512 APIs. This does not change the `CryptHash` bit-length convention, `pad10*1`, or capacity feed-forward ordering. Every path uses the new D_h , including the short-message fast path, long-message path, and partial-byte finalization. The public message-length API remains 64-bit on the tested platforms; its existing guard accepts at most $2^{64} - 3$ bits.

3 Implementation Efficiency

The AVX2 implementation processes one message at a time. Eight 256-bit YMM vectors hold the 2048-bit state in a row-oriented layout, and the four 64-bit lanes contain words from four distinct MixQuad columns of the same message. After two initial 4×4 transpositions, this layout remains in registers throughout all 16 rounds. Packed additions and XORs use AVX2 integer instructions. Most rotations use two shifts and an OR, while the 56-bit rotation uses a byte shuffle. Short messages use a single-rate-block fast path, and longer messages are absorbed without converting the state layout between rounds.

The measurements below were collected for the current submitted sources on server 241, which uses an Intel Xeon Silver 4416+ (20 cores and 40 hardware threads), Ubuntu 22.04.5, GCC 11.4.0, and CMake 3.22.1. Each process was pinned to logical CPU 2. One complete benchmark pass per binary was used for warm-up and excluded from the results. Every reported value is the median of 101 samples, and throughput is reported in decimal MB/s.

Instance	AVX2 32 B cycles/Byte	Reference 65536 B cycles/Byte	AVX2 65536 B cycles/Byte	AVX2 65536 B MB/s	Reference-to-AVX2 speedup
Eijen-256	12.456	3.464	1.581	1265.41	2.19×
Eijen-384	12.268	3.981	1.701	1175.73	2.34×
Eijen-512	12.151	4.059	1.855	1077.97	2.19×
Eijen-768	12.147	5.229	2.233	895.85	2.34×
Eijen-1024	16.375	6.650	2.841	703.88	2.34×